1. Model Development and Optimization (Hard)

Scenario: You need to train a deep learning model with a very large dataset that doesn't fit into memory. How would you handle data loading and preprocessing in both TensorFlow and PyTorch?

---

2. Machine Learning Model Integration (Medium)

Integrate a machine learning model (e.g., trained using TensorFlow or Scikit-learn) into a Python application. Implement APIs for model inference, input data preprocessing, and result interpretation. Demonstrate how the model can be used for tasks like classification, regression, or anomaly detection.

---

3. FastAPI REST Endpoint (Medium)

Write a FastAPI-based REST endpoint that accepts a JSON payload, validates input using Pydantic, authenticates the request using a simple token-based mechanism, and asynchronously calls an external API. The endpoint should handle errors, implement retries, and return a structured response.

---

4. High-Performance REST API Design (Medium)

How would you design a high-performance REST API using FastAPI or Flask that handles authentication, rate limiting, and request validation at scale? Explain how you would structure middleware, dependency injection, and background tasks while ensuring low latency and high throughput.

---

5. End-to-End RAG System Design (Medium)

How would you design an end-to-end Retrieval-Augmented Generation (RAG) or agentic AI system using tools like OpenTelemetry and Datadog or Grafana? OpenTelemetry should trace across components (Retriever, LLM calls, tool invocation). Explain how you would instrument traces, define meaningful metrics (latency, token usage, retrieval success), and correlate logs, traces, and metrics.

---

6. Distributed AI System Deployment (Medium)

In a distributed AI system deployed on cloud (AWS/GCP/Azure), how would you use tools like CloudWatch, Splunk, or Grafana to detect and diagnose intermittent failures or latency spikes in LLM responses? Discuss your approach to setting up alerts, dashboards, anomaly detection, and root-cause analysis across services such as APIs, vector databases, and model endpoints.

1. LLM Performance & Cost Optimization (Hard)

> A production AI application experiences increasing latency and escalating LLM costs as user traffic grows. Describe the techniques you would implement to optimize performance, including semantic caching, prompt caching, batching, streaming, model routing, token reduction, parallel execution, and fallback strategies while maintaining response quality.

---

2. Multi-LLM Integration (Medium)

> How would you integrate multiple LLMs such as LLaMA, Mistral, and proprietary APIs into a single AI platform? Explain model routing, fallback mechanisms, cost optimization, performance benchmarking, and dynamic model selection strategies.

---

3. Model Selection & Class Imbalance (Medium)

> You need to build a classification model but have multiple algorithms to choose from. Describe how you would evaluate the models and select the best one. What metrics would you consider and how would you handle class imbalance in your dataset?

---

4. Decision Tree Implementation (Medium)

> Write a Python program to build a Decision Tree classifier from scratch for a binary classification problem. Implement feature selection using either Gini Impurity or Information Gain, support recursive tree construction, and include prediction functionality for unseen samples.

Evaluation:

> Tree-based algorithm knowledge, recursion, data structure design, impurity calculations, optimization techniques, and ability to explain overfitting control mechanisms such as pruning and depth constraints.

---

5. SQL Query Optimization (Medium)

> Client has raised a ticket to optimize the existing SQL query. What all scenarios would you think to optimize the query?

---

6. Employees Above Department Average Salary (Medium)

> Problem: Given table Employees(emp_id, name, hired_date, dept_no, salary), find employees earning more than their department average.

These are the questions visible in the images you uploaded. I have not added or changed any question text beyond minor formatting for readability.